

AI IA -2 Filtered Questions

Note – some questions may be missing or inaccurate

UNIT 2: Intelligent Agents

Q1. Define Problem-Solving Agent. (4 Marks)

- A problem-solving agent is a type of **goal-based agent** that plans a sequence of actions to reach a specific goal state.
- It operates in **simple, fully observable environments** where goals are clearly defined and actions are predictable.
- The agent works in three main steps: **formulate the problem**, **search for a solution**, and **execute the actions** step by step.
- It uses **search algorithms** (like BFS or DFS) to explore possible paths from the initial state to the goal state.
- Once a solution is found, the agent **executes the planned actions** one at a time until the goal is achieved.

Q2. What are the components of a problem in problem solving? (4 Marks)

- **Initial State** – The starting point of the agent. Example: Agent is in City A and wants to reach City B.
- **Actions** – The set of possible moves the agent can take from any given state. Example: Move left, right, up, down.
- **Transition Model** – Defines what state results after performing an action, described as $RESULT(s, a)$.
- **Goal Test** – A condition that checks whether the current state is the desired goal state. Example: Is the agent in City B?
- **Path Cost** – A measure of the total cost of reaching the goal, used to find the most efficient solution.

Q3. Explain the concept of a Problem-Solving Agent. (5 Marks)

- A problem-solving agent is a **goal-based AI agent** that decides actions by searching for a sequence of steps leading to the goal.
- It works in **deterministic, fully observable environments** where the outcome of every action is known in advance.
- The agent first performs **goal formulation** – deciding what it wants to achieve (e.g., reach a destination).
- Next, it performs **problem formulation** – defining the initial state, possible actions, transition model, goal test, and path cost.
- It then uses **search algorithms** to explore different paths through the state space and find a solution.
- Once the search finds a valid path, the agent **executes the actions** one by one until the goal is reached.
- If no solution is found during search, the agent returns **failure** and takes a null action.

Q4. Explain the steps involved in formulating a problem for AI search. (5 Marks)

- **Goal Formulation** – The first step where the agent defines what it wants to achieve. Example: Reach City B from City A.
- **Initial State** – Define where the agent starts. Example: Agent is currently in City A.
- **Actions** – List all possible moves available from any state. Example: Drive to City C or drive to City D from City A.
- **Transition Model** – Define how each action changes the current state using RESULT(s, a). Example: Driving to City C changes state to "In City C."
- **Goal Test** – Define a condition to check if the goal is reached. Example: Is the agent in City B?
- **Path Cost** – Assign a cost to each path to help find the most optimal solution. Example: Shortest distance or minimum travel time.

Q5. Describe one example problem solved using search – 8-Puzzle Problem. (5 Marks)

- The **8-puzzle problem** consists of a 3×3 grid with 8 numbered tiles and one blank space. The goal is to rearrange tiles to match a goal configuration.
- **Initial State** – Any arrangement of 8 tiles with one blank space. Example: Tiles arranged randomly in the grid.
- **Actions** – The blank space can move **up, down, left, or right**, effectively swapping with an adjacent tile.
- **Transition Model** – Each move results in a new arrangement of tiles by swapping the blank space with a neighboring tile.
- **Goal Test** – Check if the current tile arrangement exactly matches the goal configuration (e.g., tiles ordered 1–8 with blank at end).
- **Path Cost** – Each move costs 1, so the total path cost equals the number of moves taken to reach the goal.
- Search algorithms like **BFS or A*** are used to find the minimum number of moves needed to solve the puzzle.

UNIT 3: Problem Solving

Q1. Define problem solving in AI. (4 Marks)

- Problem solving in AI refers to the process where an agent **finds a sequence of actions** to move from an initial state to a desired goal state.
- It involves **formulating the problem** by defining the initial state, actions, goal, and path cost clearly.
- The agent uses **search algorithms** to explore the state space and discover a valid path to the goal.
- Problem solving is used in both **toy problems** (like 8-puzzle, vacuum world) and **real-world problems** (like route finding, VLSI design).

Q2. What is Breadth First Search (BFS)? (4 Marks)

- BFS is an **uninformed search algorithm** that explores a tree or graph level by level, starting from the root node.
- It uses a **FIFO (First In First Out) queue** to keep track of nodes that need to be expanded next.
- BFS always expands **all nodes at the current depth level** before moving to nodes at the next level.
- It is **complete** – meaning it will always find a solution if one exists, and it finds the **shallowest (minimal step) solution** first.
- **Time and Space Complexity** are both $O(b^d)$, where b is the branching factor and d is the depth of the shallowest solution.

Q3. Differentiate between BFS and DFS (any two points). (4 Marks)

- **Data Structure:** BFS uses a **FIFO Queue** to process nodes level by level, while DFS uses a **Stack (LIFO)** to go as deep as possible along one branch first.
- **Memory Usage:** BFS requires **more memory** as it stores all nodes at the current level, while DFS requires **less memory** as it only stores nodes along the current path.
- **Completeness:** BFS is **complete** and always finds the shallowest solution, while DFS may go into an **infinite loop** and is not always complete.
- **Optimality:** BFS is **optimal** when path cost is a non-decreasing function of depth, while DFS is **not optimal** as it may find a deeper solution first.

Q4. What is Depth Limited Search? (4 Marks)

- Depth Limited Search (DLS) is a variation of DFS where the search is performed only up to a **predetermined depth limit (ℓ)**.
- Nodes at the depth limit are treated as if they have **no successors**, preventing the algorithm from going deeper.
- It can fail in two ways: **Standard failure** (no solution exists) and **Cut-off failure** (no solution found within the depth limit).
- DLS is **memory efficient** like DFS, with space complexity $O(b \times \ell)$ and time complexity $O(b^\ell)$.
- It is **not optimal** even if $\ell > d$ (where d is the depth of the shallowest goal), and is **incomplete** if the solution lies beyond the depth limit.

Q5. Define Iterative Deepening DFS (IDDFS). (4 Marks)

- IDDFS is a search algorithm that **combines the advantages of BFS and DFS** by performing DFS with gradually increasing depth limits.
- It starts with a depth limit of 0 and **increases the limit by 1 after each iteration** until the goal node is found.
- It provides the **memory efficiency of DFS** (only storing one path at a time) while also finding the **shallowest solution like BFS**.
- It is **complete** when the branching factor is finite, and **optimal** when path cost is a non-decreasing function of depth.
- The main drawback is that it **repeats work from previous iterations**, re-expanding earlier nodes in each new pass.

Q6. What is an informed search algorithm? (4 Marks)

- An informed search algorithm uses **additional knowledge** (heuristics) about the problem to guide the search more efficiently toward the goal.
- This knowledge includes information like **how far we are from the goal**, the path cost so far, and how to reach the goal node.
- It uses a **heuristic function $h(n)$** that estimates the cost from the current node to the goal. The value of $h(n)$ is always positive.
- Informed search explores **fewer nodes** compared to uninformed search, making it faster and more efficient for large search spaces.
- Common examples of informed search algorithms include **Greedy Best First Search** and **A* Search**.

Q7. Define Greedy Best First Search. (4 Marks)

- Greedy Best First Search is an **informed search algorithm** that always selects the node which appears to be **closest to the goal** at each step.
- It uses only the **heuristic function** as the evaluation function: **$f(n) = h(n)$** , where $h(n)$ is the estimated cost from node n to the goal.
- It is implemented using a **priority queue**, where nodes with the **lowest $h(n)$ value** are given the highest priority.
- Greedy Best First Search is **not complete** (can get stuck in loops) and **not optimal** (does not guarantee the shortest path).
- Its worst-case time and space complexity are both **$O(b^m)$** , where m is the maximum depth of the search space.

Q8. What is the evaluation function in A* search? (4 Marks)

- The evaluation function in A* search is $f(n) = g(n) + h(n)$, which estimates the total cost of the cheapest solution passing through node n .
- $g(n)$ is the **actual cost** from the start node to the current node n along the path taken so far.
- $h(n)$ is the **heuristic estimated cost** from node n to the goal – it must be admissible (never overestimates the true cost).
- The **sum $f(n)$** represents the total estimated cost from start to goal via node n , helping A* choose the most promising path.
- A* is both **complete and optimal** when the heuristic is admissible, making it one of the most widely used search algorithms.

Q9. Define Hill Climbing Algorithm. (4 Marks)

- Hill Climbing is a **local search algorithm** that starts from an initial state and iteratively moves to the **best neighboring state** according to an objective function.
- At each step, it selects the **successor with the highest value** (or lowest cost) and moves to it, ignoring all other paths.
- It **terminates** when it reaches a state (peak) where no neighbor has a better value than the current state.
- Hill Climbing is sometimes called **greedy local search** because it grabs the best immediate neighbor without looking ahead.
- It does **not maintain a search tree** and uses very little memory, only storing the current state and its objective value.

Q10. What is Simulated Annealing? (4 Marks)

- Simulated Annealing is a **local search algorithm** inspired by the physical process of slowly cooling a heated material to reach a low-energy state.
- Unlike Hill Climbing, it occasionally **accepts worse (downhill) moves** to escape local maxima and explore the search space more broadly.
- The probability of accepting a worse move depends on a **temperature parameter** – at high temperature, bad moves are accepted more often; as temperature decreases, fewer bad moves are accepted.
- The temperature is **gradually reduced (cooled)** over time according to a cooling schedule until the algorithm converges to a solution.
- It is particularly useful for problems with **many local optima** where hill climbing would get stuck.

Q11. Define Genetic Algorithm. (4 Marks)

- Genetic Algorithms are **search and optimization algorithms** inspired by the process of natural evolution (survival of the fittest).
- They work with a **population of candidate solutions** (called chromosomes) and evolve them over generations toward better solutions.
- Three key operations are used: **Selection** (choose the best individuals), **Crossover** (combine two parents to create offspring), and **Mutation** (randomly alter some genes to maintain diversity).
- They are useful for solving **complex optimization problems** where traditional search methods are too slow or infeasible.
- Genetic Algorithms do **not guarantee an optimal solution** but often find very good solutions efficiently across large, complex search spaces.

Now the **5 Mark questions for Unit 3:**

Q1. Explain Breadth First Search (BFS) with example. (5 Marks)

- BFS is an **uninformed search algorithm** that traverses a tree or graph **level by level**, starting from the root node and expanding all nodes at each depth before moving deeper.
- It uses a **FIFO Queue** – the first node added to the queue is the first one expanded, ensuring level-by-level exploration.
- **Advantages:** BFS always finds a solution if one exists (complete), and it finds the **minimum step solution** (optimal for uniform cost).
- **Disadvantages:** It requires **large amounts of memory** to store all nodes at each level, and takes more time if the goal is deep in the tree.
- **Example:** In a tree with root S, and goal node K at Level 4, BFS traverses: $S \rightarrow A \rightarrow B \rightarrow C \rightarrow D \rightarrow G \rightarrow H \rightarrow E \rightarrow F \rightarrow I \rightarrow K$, exploring each level fully before going deeper.
- **Time Complexity** is $O(b^d)$ and **Space Complexity** is $O(b^d)$, where b is branching factor and d is depth of shallowest solution.
- BFS is **complete** (finds a solution if one exists) and **optimal** when path cost is non-decreasing with depth.

Q2. Describe Depth First Search (DFS) and its advantages and disadvantages. (5 Marks)

- DFS is an **uninformed search algorithm** that explores a tree or graph by going as **deep as possible along one branch** before backtracking.
- It uses a **Stack (LIFO)** data structure – the most recently added node is explored first, allowing it to dive deep into the search tree.
- The algorithm starts at the root, follows a path to its **deepest node**, then backtracks to explore other paths if the goal isn't found.

- **Example:** In a tree with root S, DFS traverses: $S \rightarrow A \rightarrow B \rightarrow D \rightarrow E$ (backtracks) $\rightarrow C \rightarrow G$ (goal found), exploring depth-first along each branch.
- **Advantages:** DFS uses **very little memory** as it only stores nodes along the current path (space complexity $O(bm)$), and can reach the goal quickly if it's on the chosen branch.
- **Disadvantages:** DFS is **not optimal** and may find a longer solution first; it can get stuck in **infinite loops** in graphs with cycles if visited states aren't tracked.
- **Time Complexity** is $O(n^m)$ and **Space Complexity** is $O(bm)$, where m is the maximum depth and b is the branching factor.

Q3. Compare DFS and Depth Limited Search. (5 Marks)

- **Basic Concept:** DFS explores as deep as possible without any restriction, while Depth Limited Search (DLS) is DFS with a **predetermined depth limit (ℓ)** beyond which it will not explore.
 - **Infinite Loop Problem:** DFS can go into an **infinite loop** in infinite state spaces, while DLS avoids this by stopping at the set depth limit.
 - **Completeness:** DFS is complete only in finite state spaces, while DLS is complete only if the **solution exists above the depth limit** ($\ell \geq d$).
 - **Failure Conditions:** DFS has only standard failure (no solution), while DLS has two failures – **standard failure** (no solution) and **cut-off failure** (solution exists but is beyond depth limit).
 - **Time Complexity:** DFS has $O(b^m)$ and DLS has **$O(b^\ell)$** , where ℓ is the depth limit – DLS saves time by not exploring beyond ℓ .
 - **Space Complexity:** DFS has $O(bm)$ while DLS has **$O(b \times \ell)$** – both are memory efficient compared to BFS.
 - **Optimality:** Neither DFS nor DLS is optimal – both may find a suboptimal solution before finding the shortest one.
-

Q4. Explain Iterative Deepening Depth First Search (IDDFS). (5 Marks)

- IDDFS is a search algorithm that **combines DFS and BFS** by running DFS repeatedly with increasing depth limits (0, 1, 2, 3...) until the goal is found.
 - At each iteration, it performs a **complete DFS up to the current depth limit**, then increases the limit by 1 and starts again.
 - **Example:** For a tree with goal G at Level 2, iterations are: 1st → A; 2nd → A, B, C; 3rd → A, B, D, E, C, F, G (goal found at 4th iteration in the notes example).
 - **Advantage:** It combines the **fast search of BFS** (finds shallowest solution) with the **memory efficiency of DFS** (only stores one path at a time).
 - **Disadvantage:** The main drawback is that it **repeats all previous work** in each new iteration – nodes at shallower depths are re-expanded multiple times.
 - It is most useful when the **search space is large** and the depth of the goal node is **unknown** in advance.
 - **Completeness:** Complete if branching factor is finite. **Optimality:** Optimal if path cost is non-decreasing with depth. **Space Complexity:** $O(bd)$, **Time Complexity:** $O(b^d)$.
-

Q5. Describe Greedy Best First Search with suitable example. (5 Marks)

- Greedy Best First Search is an **informed search algorithm** that always expands the node with the **lowest heuristic value $h(n)$** – the node that appears closest to the goal.
- The evaluation function is **$f(n) = h(n)$** , where $h(n)$ is the estimated cost from node n to the goal (not the cost already incurred).
- It uses a **priority queue (OPEN list)** and a **CLOSED list** to manage which nodes to expand next and which have already been visited.
- **Algorithm Steps:** Place start node in OPEN → Select node with lowest $h(n)$ → If goal, stop → Else expand and add successors to OPEN → Repeat.

- **Example:** In a graph with nodes S, A, B, E, F, G and heuristic values $S=13, B=4, F=2, G=0$: OPEN starts with [S] → Expand S → Select B ($h=4$) → Expand B → Select F ($h=2$) → Expand F → Select G ($h=0$) → Goal found. Path: $S \rightarrow B \rightarrow F \rightarrow G$.
 - **Disadvantages:** Not complete (can get stuck in loops), not optimal (doesn't guarantee shortest path), and performance heavily depends on the **quality of the heuristic function**.
 - **Time and Space Complexity:** Both $O(b^m)$ in the worst case, where m is the maximum depth of the search space.
-

Q6. Explain A* search algorithm with evaluation function. (5 Marks)

- A* search is a **best-first informed search algorithm** that combines actual path cost and heuristic estimate to find the optimal path to the goal.
 - The evaluation function is $f(n) = g(n) + h(n)$, where $g(n)$ is the cost from start to node n , and $h(n)$ is the estimated cost from n to the goal.
 - At each step, A* selects the node from the OPEN list with the **lowest $f(n)$ value** for expansion, ensuring it explores the most promising paths first.
 - **Algorithm:** Start node added to OPEN with $f = g + h$ → Select lowest $f(n)$ node → If goal, stop → Expand and compute f for successors → Move expanded node to CLOSED → Repeat.
 - **Example:** In a graph $S \rightarrow A \rightarrow C \rightarrow G$ with h values $S=5, A=3, C=2, G=0$, A* finds path $S \rightarrow A \rightarrow C \rightarrow G$ with total cost 6, which is optimal compared to direct path $S \rightarrow G$ with cost 10.
 - **Advantages:** A* is **complete** (finds solution if one exists) and **optimal** (finds least cost path) when the heuristic is **admissible** (never overestimates the true cost).
 - **Disadvantages:** High **time and space complexity** $O(b^d)$ as it stores all generated nodes in memory; performance depends heavily on heuristic quality.
-

Q7. Explain Hill Climbing algorithm and its limitations. (5 Marks)

- Hill Climbing is a **local search algorithm** that starts from an initial state and at each step moves to the **best neighboring state** (highest value or lowest cost) according to an objective function.
 - It does **not maintain a search tree** – it only stores the current state and the value of the objective function, using very little memory.
 - The algorithm **terminates** when it reaches a peak – a state where no neighbor has a better value than the current state (local or global maximum).
 - It is called **greedy local search** because it always grabs the best immediate neighbor without thinking about future states.
 - **Limitation 1 – Local Maximum:** The algorithm may get stuck at a **local maximum** (a peak better than all neighbors but not the best overall), unable to move to the global maximum.
 - **Limitation 2 – Plateau:** It may get lost on a **plateau** (flat area of the search space) where all neighbors have the same value and no uphill move exists.
 - **Limitation 3 – Ridge:** On a **ridge** (a slope that rises in a direction not directly reachable), the algorithm may struggle to make progress.
 - **Example (TSP):** Starting with route A→B→C→D→E→A (cost 29), hill climbing improves to A→B→D→C→E→A (cost 26) by swapping cities, stopping when no swap improves the route.
-

Q8. Describe Simulated Annealing with working principle. (5 Marks)

- Simulated Annealing is a **probabilistic local search algorithm** inspired by the annealing process in metallurgy, where a material is heated and then slowly cooled to reduce defects.
- Unlike Hill Climbing, it can **accept worse solutions occasionally** to escape local optima, giving it a better chance of finding the global optimum.
- **Working Principle:** At each step, a random neighbor is generated. If it's better, it is accepted. If it's worse, it is accepted with a probability $e^{(\Delta E/T)}$, where ΔE is the change in quality and T is the current temperature.
- At **high temperature**, the algorithm accepts bad moves more often (exploring widely). As temperature **decreases**, it becomes more selective and converges toward a good solution.
- The temperature is reduced according to a **cooling schedule** (e.g., $T = T \times 0.99$ at each step) until it reaches near zero.
- It is particularly effective for problems with **many local maxima** like the Travelling Salesman Problem, where hill climbing would easily get trapped.
- **Advantage:** Can find near-optimal solutions in complex search spaces. **Disadvantage:** Slow convergence and requires careful tuning of the cooling schedule.

Q9. Explain the basic concept of Genetic Algorithms. (5 Marks)

- Genetic Algorithms (GA) are **evolutionary search and optimization algorithms** inspired by Darwin's theory of natural selection – survival of the fittest.
- They work with a **population of candidate solutions** (called chromosomes or individuals), evolving them over multiple generations to improve the solution quality.
- **Selection:** The best-performing individuals (based on a fitness function) are selected to be **parents** for the next generation – better solutions have a higher chance of being chosen.
- **Crossover (Recombination):** Two selected parents **exchange portions** of their chromosomes to create new offspring, combining the strengths of both parents.
- **Mutation:** Random changes are made to some genes in an offspring to **introduce diversity** and prevent the algorithm from getting stuck in local optima.
- The process repeats over many generations, with the population gradually **evolving toward better solutions** according to the fitness function.
- **Advantages:** GAs work well for **large, complex, non-linear search spaces** where traditional methods fail. **Disadvantage:** They don't guarantee an optimal solution and require significant computation over many generations.